Diseño de una base de datos documental para elementos transponibles con MongoDB y Python

Milerick Henao - Manuel Noreña - Oswaldo Daza

29 Octubre

Abstract

Antecedentes. Los elementos transponibles (TEs) se cuentan entre los componentes más abundantes de los genomas eucariotas, y su estudio combina datos de naturaleza muy distinta: secuencias, una clasificación en tres niveles, la taxonomía de cada especie y perfiles de k-mers de hasta 5 460 valores por secuencia. Recursos como PanTEon distribuyen esta información en archivos planos unidos solo por el nombre de la especie, sin consultas ni relaciones explícitas. El objetivo de este trabajo fue diseñar, implementar y evaluar una base de datos documental en MongoDB que integre esas fuentes.

Resultados. Un *pipeline* reproducible en Python leyó y validó 239 946 secuencias de 2 790 especies de Animalia, Plantae y Fungi, corrigió seis problemas de calidad del FASTA (entre ellos secuencias en minúsculas, nombres de especie que no cruzaban con el CSV y 22,9 % de secuencias con bases indeterminadas) sin descartar ningún registro, y calculó los perfiles de k-mers de $k = 1$ a 6 (1 439 676 documentos). El modelo usa tres colecciones relacionadas por referencias, seis índices justificados por consultas concretas y validación `$jsonSchema` en todas las colecciones. Las consultas mostraron una base dominada por retrotransposones LTR (49,7 %), con perfiles distintos por reino (LTR en plantas, LINE en animales), y revelaron que 45 entradas registradas como especie son en realidad taxones superiores. Al comparar dos formas de guardar los k-mers en una muestra de 5 000 secuencias, una colección aparte y una colección embebida tuvieron tiempos de carga sin diferencia clara (2,7 frente a 2,3 s) y un espacio en disco similar (42,6 frente a 46,0 MB); con índices, la colección aparte respondió una consulta sobre los perfiles en 14 ms frente a 148 ms.

Conclusiones. Un modelo documental con referencias permite integrar secuencias, taxonomía y perfiles de k-mers sin repetir información y responder las consultas con agregaciones. Para consultas que trabajan con un valor de k a la vez, guardar los k-mers en una colección aparte e indexada es la opción más adecuada.

Palabras clave: elementos transponibles, MongoDB, bases de datos documentales, k-mers, bioinformática.

1 Introducción

Los elementos transponibles (TEs, por sus siglas en inglés) son secuencias de ADN capaces de moverse o de copiarse dentro del genoma que las hospeda [1]. Constituyen una fracción considerable de muchos genomas eucariotas: cerca de la mitad del genoma humano deriva de ellos [2] y en el maíz representan alrededor del 85 % de la secuencia [3]. Lejos de ser “ADN

basura", los TEs participan en la evolución de los genomas, en la regulación de la expresión génica y en la generación de variación genética sobre la que actúa la selección [1].

Para estudiarlos, los TEs se organizan en una clasificación jerárquica. El sistema propuesto por Wicker et al. [4] los divide en dos clases según su mecanismo de transposición: la Clase I, o retrotransposones, que se movilizan a través de un intermediario de ARN, y la Clase II, o transposones de ADN. Cada clase se subdivide en órdenes (por ejemplo, LTR, LINE o TIR) y estos, a su vez, en superfamilias (por ejemplo, *Gypsy*, *Copia* o *hAT*). Esta jerarquía Clase/Orden/Superfamilia es la que siguen las etiquetas de la base de datos utilizada en este trabajo [5].

El análisis de los TEs plantea retos tanto biológicos como computacionales. Desde el punto de vista biológico, los TEs suelen ser específicos de cada especie, divergen con rapidez incluso entre especies cercanas y, dentro de una misma familia, sus copias van desde casi idénticas hasta muy degradadas; además, su alto número de copias y las inserciones anidadas dificultan su detección y anotación [5]. Desde el punto de vista de los datos, la información sobre TEs es voluminosa, heterogénea y poco estructurada: las secuencias se distribuyen en archivos FASTA cuyo encabezado mezcla en texto libre el identificador, la clasificación y la especie, mientras que la taxonomía de cada especie y el origen de cada biblioteca se describen en archivos de metadatos aparte [5, 6]. A ello se suma la transformación habitual de las secuencias en perfiles de k-mers, una de las representaciones más usadas para entrenar clasificadores de TEs [5]: con el alfabeto $\{\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T}\}$ existen 4^k k-mers posibles, de modo que el perfil completo para $k = 1, \ldots, 6$ tiene hasta 5 460 valores por secuencia. Para las 239 946 secuencias de PanTEon v1.6.1 esto supone del orden de $1{,}3 \times 10^9$ valores, lo que convierte la forma de almacenarlos y consultarlos en una decisión de diseño central.

En este trabajo diseñamos, implementamos y evaluamos una base de datos documental en MongoDB que integra las secuencias de TEs de PanTEon Database v1.6.1 [6], la taxonomía de las especies de las que proceden y sus perfiles de k-mers ($k = 1$ a 6). El procesamiento se realiza con un *pipeline* reproducible en Python que lee y valida las secuencias, documenta y resuelve los problemas de calidad de las fuentes, cruza ambas fuentes por especie y carga los datos de forma masiva. Además, respondemos un conjunto de preguntas de investigación mediante consultas y agregaciones, y comparamos experimentalmente dos estrategias de almacenamiento de los k-mers en términos de tiempo de carga, espacio en disco y tiempo de consulta.

Trabajo relacionado

PanTEon. PanTEon [5] es un marco de trabajo para diseñar y comparar clasificadores de TEs basados en aprendizaje profundo. Combina una base de datos de TEs curada automáticamente, que abarca los reinos Animalia, Plantae y Fungi, con una plataforma que estandariza el entrenamiento, la evaluación y la inferencia de nueve métodos de aprendizaje automático. Su versión 1.6.1 reúne 239 946 TEs de 2 790 especies. Las secuencias proceden de dos orígenes: familias de Dfam y bibliotecas generadas *de novo* con RepeatModeler2 [7] y curadas automáticamente con MCHelper [8]; solo se conservaron las secuencias con todos los rasgos estructurales esperados para su superfamilia. Cada secuencia se etiqueta con la jerarquía Clase/Orden/Superfamilia. Los datos se distribuyen en dos archivos planos: un FASTA con las secuencias y un CSV con la taxonomía y el origen de cada biblioteca, relacionados únicamente por el nombre de la especie [6]. Esta organización es adecuada para entrenar modelos, pero no ofrece consultas, índices ni relaciones explícitas entre secuencias y taxonomía, y no almacena los perfiles de k-mers, que cada usuario debe recalcular.

Dfam. Dfam [9] es un recurso comunitario de familias de TEs y otros repetidos. Cada familia se construye a partir de un alineamiento semilla (*seed alignment*) de copias representativas, del que se derivan un modelo oculto de Markov de perfil (HMM) y una secuencia consenso; ambos se usan para anotar genomas. Las familias se dividen en curadas, con umbrales de tasa de falsos positivos caracterizados, y no curadas, identificadas con el prefijo de acceso "DR". Cada familia se asocia a la taxonomía del NCBI, y la clasificación se representa como un árbol no jerarquizado por rangos cuyos nombres se concatenan desde la raíz (por ejemplo, `Interspersed_repeat;Unknown`), con correspondencias hacia los esquemas de RepeatMasker, Repbase y Wicker et al. Dfam se consulta mediante un portal web y una API, y se distribuye también en FamDB, un formato basado en HDF5 que permite consultas locales. En la versión descrita por Storer et al. [9] contenía 273 655 familias.

Diferencia con este trabajo. Dfam está orientado a la anotación de genomas y representa cada familia mediante modelos (HMM y consenso), mientras que PanTEon está orientado al entrenamiento de clasificadores y distribuye las secuencias en archivos planos. Ninguno de los dos almacena perfiles de k-mers consultables junto con la secuencia y la taxonomía de la especie. Este trabajo aborda esa necesidad mediante un modelo documental que permite recorrer la relación secuencia–especie en ambos sentidos, consultar los k-mers de cualquier secuencia y evaluar el costo de distintas formas de almacenarlos.

2 Methods

Modelo de datos

Los datos de partida son dos archivos. El FASTA contiene las secuencias de los elementos transponibles (TEs) y, en el encabezado de cada una, su clasificación y la especie. El CSV de metadatos contiene la taxonomía de cada especie (reino, filo, clase, orden y familia), el origen de su biblioteca de TEs y algunas notas. El único dato que comparten los dos archivos es el nombre de la especie, por lo que se usó como llave para unirlos.

Con esta información se construyó una base de datos en MongoDB con tres colecciones: `especies`, `secuencias` y `kmers` (Figura 1). Las colecciones no están anidadas: se conectan por referencias, es decir, un documento guarda el `_id` de otro.

Colección `especies`. Tiene un documento por especie (2 790), que viene del CSV. Guarda la taxonomía (`reino`, `filo`, `tax_clase`, `tax_orden` y `familia`), el nombre de la especie y el origen de su biblioteca: `fuente` (Dfam o RM2+MCH), `fuente_2` y `notas`, estos dos últimos tal como vienen en el CSV. La clase y el orden de la taxonomía llevan el prefijo `tax_` para distinguirlos de la clase y el orden del TE.

Colección `secuencias`. Tiene un documento por TE (239 946). Del encabezado del FASTA se toman el identificador original (`pdb_id`) y los tres niveles de la clasificación (clase, orden y superfamilia), cada uno en un campo aparte. También se guarda la secuencia, en mayúsculas, y tres valores calculados al leerla: longitud, porcentaje de GC y porcentaje de N. Se guardan ya calculados para que las consultas que solo necesitan la longitud, como la distribución de longitudes por orden, no tengan que leer la secuencia. La especie no se copia: cada secuencia guarda `especie_id`, que apunta al documento de su especie.

Colección `kmers`. Guarda el resultado del conteo de k-mers para k de 1 a 6, con un documento por secuencia y por valor de k (seis por secuencia, 1 439 676 en total si todas las secuencias están procesadas). Cada documento tiene la referencia a la secuencia (`secuencia_id`), el valor de k,

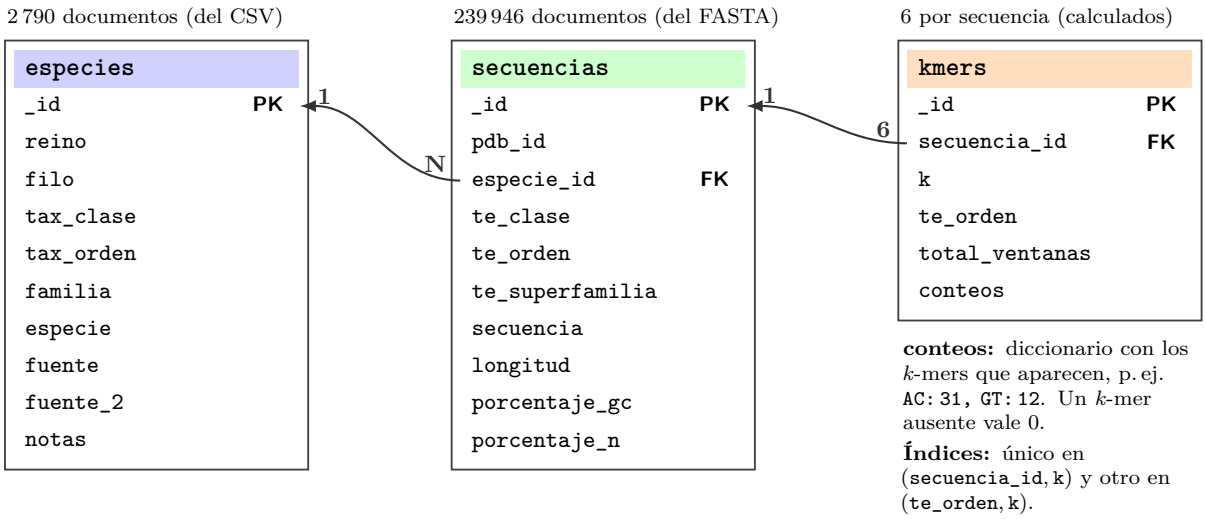


Figure 1. Modelo de datos en MongoDB. Cada recuadro es una colección y cada fila un campo. `PK` marca el identificador del documento y `FK` un campo que guarda el `_id` de otra colección (flecha). Una especie tiene muchas secuencias (1:N) y cada secuencia tiene seis documentos de k-mers, uno por cada k de 1 a 6.

`total_ventanas` (ventanas válidas contadas) y `conteos`, un diccionario con cada k-mer y su número de apariciones. Solo se guardan los k-mers que aparecen, de modo que un k-mer ausente tiene conteo cero y no es un dato faltante. También se copia `te_orden` para filtrar por orden de TE sin pasar por `secuencias`. La colección tiene dos índices: uno único en (`secuencia_id`, k), que evita documentos repetidos, y otro en (`te_orden`, k), para las consultas y figuras por orden.

Referencias y no copias. Se pudo copiar la taxonomía completa dentro de cada secuencia, pero eso habría repetido los mismos datos 239 946 veces y una corrección habría obligado a cambiar miles de documentos. Con la referencia, la taxonomía está en 2 790 documentos y una corrección es un solo cambio. También simplifica algunas consultas: para contar TEs por especie se agrupa primero (18 513 grupos de especie y clasificación) y después se une con `especies`, en lugar de unir los 239 946 documentos. La desventaja es que otras consultas necesitan un `$lookup` para juntar las colecciones.

Decisiones de diseño

Entidades y colecciones. Identificamos tres entidades, la especie, la secuencia y el perfil de k-mers, y cada una quedó en su propia colección. La colección de especies tiene un documento por cada fila del CSV, con su taxonomía y el origen de su biblioteca de TEs, mientras que las secuencias y los k-mers se guardan aparte porque son muchísimos más y tienen un tamaño muy distinto. Decidimos separarlas así porque una especie tiene muchas secuencias (hasta 1 627 en una sola especie) y mezclarlas en un solo documento habría hecho que cada consulta tuviera que cargar mucha información que no necesita.

Información embebida y referenciada. La taxonomía de cada especie se guarda una sola vez en la colección de especies y las secuencias solo guardan el identificador de su especie (`especie_id`), de modo que se llega a la taxonomía haciendo un `$lookup`. Esto porque embeber la taxonomía en cada secuencia habría repetido los mismos datos cerca de 240 000 veces, una por cada secuencia, y cualquier corrección de un nombre habría que hacerla en todos los documentos de esa especie en vez de uno. Otra opción era embeber toda la taxonomía para evitar el `$lookup`, pero la ganancia en las consultas por reino no nos pareció suficiente para justificar esa repetición.

El costo de la referencia es que las consultas que agrupan por reino o por filo necesitan unir las dos colecciones, y los identificadores que genera MongoDB cambian cada vez que se recarga la colección de especies, por lo que las secuencias también hay que volver a cargarlas.

Jerarquías de clasificación y de taxonomía. No se representa la jerarquía en los datos porque no creímos que fuera algo que fuera importante almacenar, solo se pide que se pueda responder a consultas y para ello no es necesaria una estructura jerárquica, así que se dejó implícita. Los campos sin embargo están ordenados siguiendo esa jerarquía, aunque esto no tenga relevancia al final más allá de un mejor orden visual. Consideramos usar *arrays* de modo que con el índice de cada elemento se diera esta estructura, pero a la hora de hacer las consultas es mucho más sencillo agrupar utilizando campos ya que no habría que acceder a la posición.

Columnas del CSV y nombres de los campos. Solo se conservan las nueve columnas necesarias y las demás se descartan al leer el archivo, ya que no aportan nada a las consultas ni al cruce. Los nombres se pasaron al español y a `Class` y `Order` se les puso el prefijo `tax_` (`tax_clase` y `tax_orden`) porque la clase y el orden de la especie no tienen nada que ver con la clase y el orden del elemento transponible que vienen en el FASTA, y dejarlos con el mismo nombre habría sido una fuente de errores al hacer las consultas.

Limpieza de textos y comparación de nombres. A todas las columnas de texto se les quitan los espacios sobrantes al inicio, al final y entre palabras, y los valores como `NA`, `N/A` o "`-`" se convierten en vacíos. Para comparar nombres entre el CSV y el FASTA se usa una versión normalizada que pasa todo a minúsculas y quita el punto de abreviaturas como `sp.` y `subsp.`, pero esa versión solo sirve para comparar y nunca se guarda, porque un nombre científico va con el género en mayúscula y al exportar las bibliotecas el encabezado tiene que quedar igual que en el archivo original. Otra opción era guardar el nombre normalizado, pero eso habría hecho que las bibliotecas generadas ya no coincidieran con el FASTA. Con esto el cruce funcionó para las 2 790 especies, y solo 4 necesitaron la normalización (una diferencia de un punto en `subsp.` o `sp.`), que corresponden a 533 secuencias.

Valores faltantes, nombres inconsistentes y registros que no cruzan. Para tratar los datos faltantes decidimos conservar las filas que tenían campos vacíos excepto cuando falta el nombre de la especie, esto porque el nombre de la especie es el principal identificador para relacionarla con las secuencias del archivo FASTA y si este campo está vacío no es posible realizar el cruce de forma confiable, por lo que en esos casos sí descartamos la fila completa. Los demás atributos se conservan aunque no tengan ningún valor, ya que no impide identificar la especie ni procesar su secuencia.

Para los registros que no cruzan identificamos si los nombres normalizados entre ambos archivos coinciden, de ser así, se le da prioridad al nombre del archivo FASTA y se reemplaza el nombre del CSV. Decidimos hacerlo así porque de igual manera habría que elegir cuál de los dos nombres debe utilizarse, y modificar el CSV no solo es mucho más sencillo al haber menos lugares donde realizar el cambio, sino que también si alguien pasara su propio archivo de secuencias por nuestro programa para realizar alguna consulta este se vería modificado y podría acarrear problemas si llegara a utilizarse en algún otro lugar. Otra opción era simplemente ignorar estas discrepancias y no contarlas en el análisis, pero un pequeño error en un nombre tiene mucho menos peso que la pérdida de cientos de secuencias asociadas a ese nombre, por lo que decidimos darle más importancia a esa información.

Duplicados. Comprobamos que el número de filas del CSV coincide con el número de especies distintas después de normalizar (2 790 en ambos casos), por lo que no hay especies repetidas y no fue necesario fusionar filas. Decidimos no escribir una lógica de fusión porque no habría tenido nada que fusionar y solo habría añadido código que nadie usa, pero la comprobación se mantiene en el reporte para que, si alguien cambia el CSV, el duplicado se note en vez de perderse. Dejar que se elija la primera fila de cada grupo habría sido más corto, pero habría descartado datos sin avisar.

Lectura del FASTA para el cruce. Se recorre el archivo línea por línea y solo se miran los encabezados, ya que el FASTA pesa 947 MB y cargarlo completo en memoria no era viable ni necesario, porque para el cruce solo hace falta el nombre de la especie. El costo es que el archivo se tiene que recorrer otra vez cuando se cargan las secuencias, pero es pequeño comparado con el riesgo de quedarnos sin memoria. Además, si una especie aparece escrita de dos formas distintas en el FASTA el programa se detiene en vez de elegir una en silencio, porque esa discrepancia no se puede resolver de forma automática y confiable.

Repetición del *pipeline*. Antes de insertar se vacía la colección de especies, así que el *pipeline* se puede ejecutar varias veces y siempre deja el mismo resultado. Otra opción era insertar solo lo nuevo, pero con 2 790 filas es mucho más simple borrar y volver a cargar todo, y eso evita que queden documentos viejos si el CSV cambia.

Calidad del archivo FASTA

Antes de diseñar el *pipeline* se exploró el archivo `PanTEon_Database_v1.6.1.fasta` (947 MB) [6] con herramientas de línea de comandos que lo recorren línea por línea, sin cargarlo en memoria. El archivo contiene 239 946 secuencias de 2 790 especies. Su estructura resultó consistente: todos los encabezados siguen el patrón

```
>ID#CLASE/ORDEN/SUPERFAMILIA @Especie
```

todas las clasificaciones tienen tres niveles, no hay identificadores repetidos, no hay secuencias vacías (la más corta tiene 81 pb) y el archivo usa saltos de línea Unix. Aun así, se encontraron seis problemas, resumidos en la Tabla 1.

Ningún registro se descartó: el *pipeline* cuenta los registros leídos, válidos y descartados por motivo (encabezado mal formado, secuencia vacía o caracteres fuera de ACGTN), y en la ejecución sobre el archivo completo los 239 946 registros resultaron válidos. Se decidió corregir en lugar de descartar porque cada problema afectaba a la forma de los datos y no a su contenido: eliminar las secuencias con N, por ejemplo, habría hecho perder casi una cuarta parte de la base. Además, 45 entradas del campo especie corresponden a taxones superiores (por ejemplo, Homininae o Mammalia); se conservaron y se analizan en los resultados como una limitación del modelo.

Calidad del CSV y del cruce con el FASTA

El CSV también necesitó limpieza antes de cargarse (Tabla 2). Se leyeron nueve columnas (taxonomía, especie, fuentes y notas) y sus nombres se pasaron a español para que coincidieran con los campos de la base.

Para unir los dos archivos se normalizó primero el nombre de cada especie: se pasó a minúsculas, se quitaron los espacios sobrantes y se eliminó el punto de las abreviaturas (`subsp.`, `sp.`, `var.` y `f.`). El resultado se usó como llave del cruce. Con la llave se separaron tres grupos: las especies que están en los dos archivos, las que están solo en el FASTA y las que están solo en el CSV.

Table 1. Problemas de calidad del archivo FASTA y regla aplicada en el *pipeline*.

Problema	**Alcance**	**Solución**
Secuencias en minúsculas	208 354 secuencias (86,8 %)	Conversión a mayúsculas al leer, antes de calcular GC, N y *k*-mers.
Superfamilia escrita de dos formas (`SINE/TRNA` y `SINE/tRNA`)	56 secuencias	Clasificación en mayúsculas; las superfamilias pasan de 66 a 65.
Nombres de especie con punto en las abreviaturas (`subsp.`, `sp.`), que en el CSV aparecen sin punto	4 especies, 533 secuencias	Normalización común para FASTA y CSV (eliminar el punto, espacios sobrantes y espacios múltiples). Con ella cruzan las 2 790 especies.
Bases indeterminadas (N)	54 837 secuencias (22,9 %); 2 147 161 N	Se conservan; se calcula `porcentaje_n` y el %GC se calcula sobre A, C, G y T. No hay otros códigos IUPAC: el alfabeto válido es ACGTN.
Identificadores que no siguen el formato `PDB########`	31 600 secuencias (13,2 %), de las especies de Dfam	El campo `pdb_id` guarda el identificador original sin asumir un formato; el índice único se mantiene porque no hay repetidos.
Superfamilia igual al orden (`LINE/LINE`, `TIR/TIR`, etc.)	11 superfamilias, 21 544 secuencias	Se conservan tal cual y se documentan como superfamilia no determinada.

Table 2. Problemas de calidad del CSV y del cruce con el FASTA, y regla aplicada en el *pipeline*.

Problema	**Solución**
Valores faltantes escritos de varias formas (`NA`, `N/A`, celda vacía y `-`)	Se leen los cuatro como nulos. En MongoDB se guardan como `null` y no como NaN.
Espacios al principio o al final y espacios repetidos en las columnas de texto	Se eliminan en todas las columnas de texto antes de comparar.
Filas sin nombre de especie	Se eliminan, porque no se pueden enlazar con ninguna secuencia.
Nombre de especie escrito distinto en el CSV y en el FASTA (Tabla 1)	Se compara con una llave normalizada. Si el nombre del CSV no es idéntico al del FASTA, se usa el del FASTA.
Encabezados del FASTA sin especie (sin `@` o con el nombre vacío)	Se guardan en una lista aparte y se reportan; no se borran en silencio. En la carga no quedó ninguna secuencia sin especie.

Cuando el nombre del CSV era distinto se dejó el del FASTA, de modo que el nombre guardado en `especies` es idéntico al de los encabezados. Antes de reemplazar se comprueba que cada llave tenga una sola escritura en el FASTA; si no se cumple, el programa se detiene. Con este procedimiento cruzaron las 2 790 especies.

Calidad del conteo de *k*-mers

Casi una cuarta parte de las secuencias tiene N. Para contar k-mers (k de 1 a 6) se descartó toda ventana con algún carácter distinto de A, C, G o T, en lugar de inventar la base o quitar la secuencia completa. Cada documento de `kmers` guarda `total_ventanas`, y las frecuencias se calculan dividiendo entre ese número y no entre la longitud, de modo que siguen sumando 1 aunque se hayan descartado ventanas.

Para medir la pérdida se comparó, con $k = 1$, `total_ventanas` contra la longitud de cada secuencia. Se encontraron 54 837 secuencias (22,85 %) con menos ventanas que su longitud, y en total se descartaron 2 147 161 caracteres, el 0,222 % de todas las bases. Todas las secuencias afectadas tienen `porcentaje_n` mayor que cero, lo que coincide con la revisión del FASTA: lo que se descarta son las N. La cuenta es para $k = 1$; con k mayores cada N invalida todas las ventanas que la contienen, así que la pérdida real crece con k.

El conteo se hace en paralelo y tarda, por lo que la carga se diseñó para poder repetirse sin duplicar datos: el índice único (`secuencia_id`, k) rechaza los documentos repetidos, los errores por duplicado se ignoran y cualquier otro error se reporta, y al volver a correr se omiten las secuencias que ya tienen $k = 6$. Como control final se compara el número de documentos de `kmers` con seis veces el de `secuencias`.

Índices

Cada índice se creó para una consulta concreta (Tabla 3). Además del índice por defecto sobre `_id`, que es el que usa el `$lookup` entre secuencias y especies, se crearon seis índices. Tres de ellos son únicos porque, además de acelerar la búsqueda, garantizan la integridad: no puede haber dos especies con el mismo nombre, dos secuencias con el mismo identificador ni dos perfiles del mismo k para una misma secuencia, lo que hace que una carga repetida no duplique datos.

Table 3. Índices creados y consulta que justifica cada uno.

Colección	Campos	Único	Consulta que lo justifica
`especies`	`especie`	Sí	Cruce FASTA–CSV y bibliotecas por especie (Q9, Q10).
`especies`	`fuente`	No	Comparación por origen de los datos, Dfam frente a RM2+MCH (Q13).
`secuencias`	`pdb_id`	Sí	Búsqueda de una secuencia por su nombre (Q8).
`secuencias`	`especie_id`	No	`$lookup` entre secuencias y especies (Q2–Q4, Q12) y secuencias de una especie (Q9–Q11).
`kmers`	`secuencia_id`, `k`	Sí	Perfil de una secuencia para un k dado (Q15).
`kmers`	`te_orden`, `k`	No	Promedio de 2-mers por orden (Figura 4).

No se crearon índices sobre `reino`, `filo` ni `familia` porque la colección de especies tiene apenas 2 790 documentos y recorrerla completa es casi instantáneo, así que un índice extra solo

ocuparía espacio sin que se notara la mejora. Tampoco se indexaron `te_clase`, `te_orden` ni `te_superfamilia` en `secuencias`. Las consultas que usan estos campos (Q1, Q5) agrupan toda la colección, y un índice no evita recorrerla; además, la distribución es tan desbalanceada que un filtro por LTR devuelve la mitad de los documentos (Q1), de modo que el índice sería poco selectivo y solo agregaría costo a cada inserción.

Validación de esquema

Siguiendo la decisión del equipo, se definió una validación `$jsonSchema` para las tres colecciones. El *pipeline* ya limpia y valida los datos antes de insertarlos, pero esa protección solo existe mientras se use el *pipeline*; el esquema, en cambio, queda en la base y rechaza cualquier documento mal formado (`validationAction = "error"`), venga de donde venga. Los campos obligatorios y las restricciones principales son:

- **`especies`**: obligatorios `especie` (texto no vacío), `reino` (solo Animalia, Plantae o Fungi) y `fuente`. Los niveles taxonómicos (`filo`, `tax_clase`, `tax_orden`, `familia`), `fuente_2` y `notas` aceptan texto o `null`, porque el CSV tiene faltantes legítimos en esos campos.
- **`secuencias`**: los nueve campos son obligatorios. `te_clase` solo admite CLASSI o CLASSII; `secuencia` debe cumplir el patrón `^[ACGTN]+$`, lo que garantiza que se guardó en mayúsculas y con el alfabeto válido; `especie_id` debe ser un `ObjectId`; `longitud` es un entero positivo, y `porcentaje_gc` y `porcentaje_n` están entre 0 y 100 (`porcentaje_gc` admite `null` para una secuencia formada solo por N).
- **`kmers`**: obligatorios `secuencia_id` (`ObjectId`), `te_orden`, `k` (entero entre 1 y 6), `total_ventanas` (entero no negativo) y `conteos` (objeto).

La validación se aplicó con `validationLevel = "moderate"`, que no revisa los documentos ya cargados; por eso, después de aplicarla se contaron los documentos que no cumplían el esquema con una consulta `$nor`/`$jsonSchema` sobre cada colección. Ninguno de los 2 790 documentos de `especies` ni de los 239 946 de `secuencias` incumplió su esquema, lo que confirma que las reglas de limpieza del *pipeline* se aplicaron a toda la base. La colección `kmers` se creó con la validación activa antes de la carga, de modo que cada documento se validó al insertarse.

3 Results

Composición de la base por clasificación (Q1)

Pregunta. ¿Cuántos TEs hay por clase, por orden y por superfamilia?

Resultado. La Clase I (retrotransposones) reúne 194 774 secuencias (81,2 %) y la Clase II (transposones de ADN), 45 172 (18,8 %). De los 11 órdenes, tres concentran el 93,4 % de la base: LTR (119 368; 49,7 %), LINE (68 382; 28,5 %) y TIR (36 471; 15,2 %). En el otro extremo, PLE, RETROPOSON, CRYPTON y MAVERICK suman menos de 630 secuencias en conjunto. Las 65 superfamilias tienen una distribución igualmente desbalanceada: *Copia* (21,2 %) y *Gypsy* (18,3 %) cubren casi el 40 % de la base, mientras que 16 superfamilias tienen menos de 100 secuencias.

Interpretación. La base está fuertemente sesgada hacia los retrotransposones LTR. Para el diseño de la base de datos esto implica que un índice sobre `te_orden` es poco selectivo para los órdenes mayoritarios: una consulta por LTR devuelve la mitad de la colección. Para el uso de la base como insumo de clasificadores [5], el desbalance indica que las clases minoritarias tendrán pocos ejemplos. Además, 11 superfamilias tienen el mismo nombre que su orden (por ejemplo, `LINE/LINE` o `TIR/TIR`; 21 544 secuencias en total), lo que en la práctica indica que la

superfamilia no se determinó; estas secuencias se conservaron, pero deben tratarse con cuidado en análisis por superfamilia.

TEs por especie y clasificación (Q2)

Pregunta. ¿Cuántos TEs tiene cada especie, desglosados por clase, orden y superfamilia?

Resultado. La consulta produce 18 513 combinaciones especie–clasificación. La diversidad de superfamilias por especie es muy variable: la mediana es de 5 superfamilias, pero *Danio rerio* alcanza 36 y 493 especies tienen una sola. Además, 652 especies tienen TEs de un único orden. Por ejemplo, en *Malus domestica*, la especie con más TEs, predominan *Copia* (671) y *Gypsy* (398).

Interpretación. La mayoría de las especies tiene bibliotecas pequeñas y poco diversas, sobre todo en Fungi, y unas pocas concentran gran parte de las secuencias. Como la especie se guarda por referencia (`especie_id`), esta consulta exige un `$lookup`; para reducir su costo se agrupa primero en `secuencias` y luego se cruza con `especies`, de modo que el `$lookup` se ejecuta sobre 18 513 grupos y no sobre 239 946 documentos.

Cambios entre reinos y phyla (Q3)

Pregunta. ¿Cómo cambia la cantidad de TEs por clase y orden entre reinos? ¿Y entre phyla?

Resultado. La composición difiere claramente entre reinos (Figura 2). En Plantae, el 72,3 % de los TEs son LTR y la Clase I alcanza el 87,7 %. En Animalia predominan los LINE (44,0 %), seguidos de LTR (25,1 %) y TIR (21,4 %). Fungi es intermedio, con LTR (40,1 %) y LINE (31,0 %), y es el reino con mayor proporción de HELITRON (8,6 %, frente a 1,2–1,9 % en los otros). RETROPOSON y CRYPTON solo aparecen en Animalia. Por phyla, Angiosperma replica el patrón de Plantae (73 % LTR), mientras que dentro de Animalia hay diferencias marcadas: Arthropoda (61 %) y Mollusca (57 %) están dominados por LINE, y Chordata está más repartido (LINE 35 %, LTR 29 %, TIR 25 %). En Fungi, Basidiomycota y Mucoromycota tienen 13 % de HELITRON.

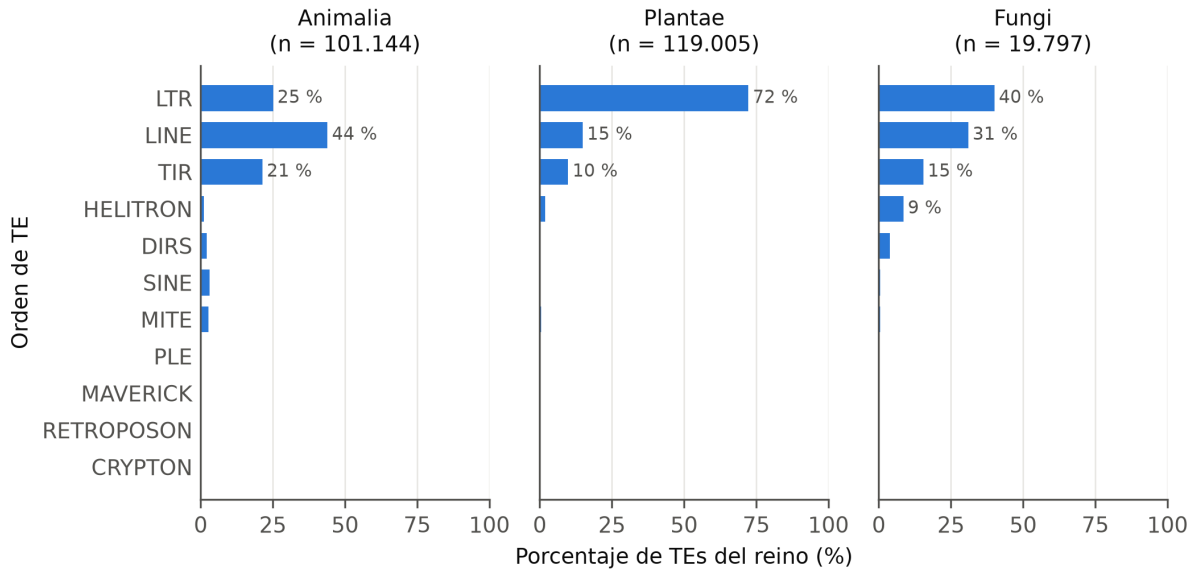


Figure 2. Composición de TEs por reino: porcentaje de cada orden de TE dentro de Animalia, Plantae y Fungi (n = número de TEs del reino). Datos obtenidos mediante agregación en MongoDB.

Interpretación. El predominio de LTR en plantas y de LINE en animales es coherente con lo

descrito para estos grupos. Sin embargo, estos porcentajes describen las bibliotecas de PanTEon (familias de TEs representadas en la base) y no la fracción del genoma que ocupa cada orden, y el número de especies por reino es muy desigual (Fungi: 1 559 especies con una mediana de 5 TEs; Plantae: 451 especies con una mediana de 245). Por eso se usan porcentajes dentro de cada reino en lugar de conteos absolutos. La consulta depende de la taxonomía del CSV: cuatro secuencias pertenecen a especies sin *phylum* registrado.

Longitud de los TEs por orden (Figura 2)

Resultado. En toda la base, la longitud de los TEs va de 81 a 19 926 pb, con una mediana de 4 259 pb (Figura 3). Los órdenes se separan en dos grupos. Los más cortos son SINE (mediana 254 pb), CRYPTON (282 pb) y MITE (346 pb), cuyas cajas quedan por debajo de 500 pb. Los más largos son MAVERICK (6 758 pb), LTR (5 347 pb; rango intercuartílico de 4 314 a 6 842 pb) y DIRS (5 143 pb). Entre ambos extremos están HELITRON (3 371 pb), LINE (3 279 pb), PLE (2 208 pb), TIR (1 341 pb) y RETROPOSON (1 159 pb). Varios órdenes muestran límites exactos: SINE y MITE no superan los 500 pb, LINE va exactamente de 500 a 6 000 pb y TIR no llega a 10 000 pb, mientras que varios órdenes tienen un mínimo de 100 pb.

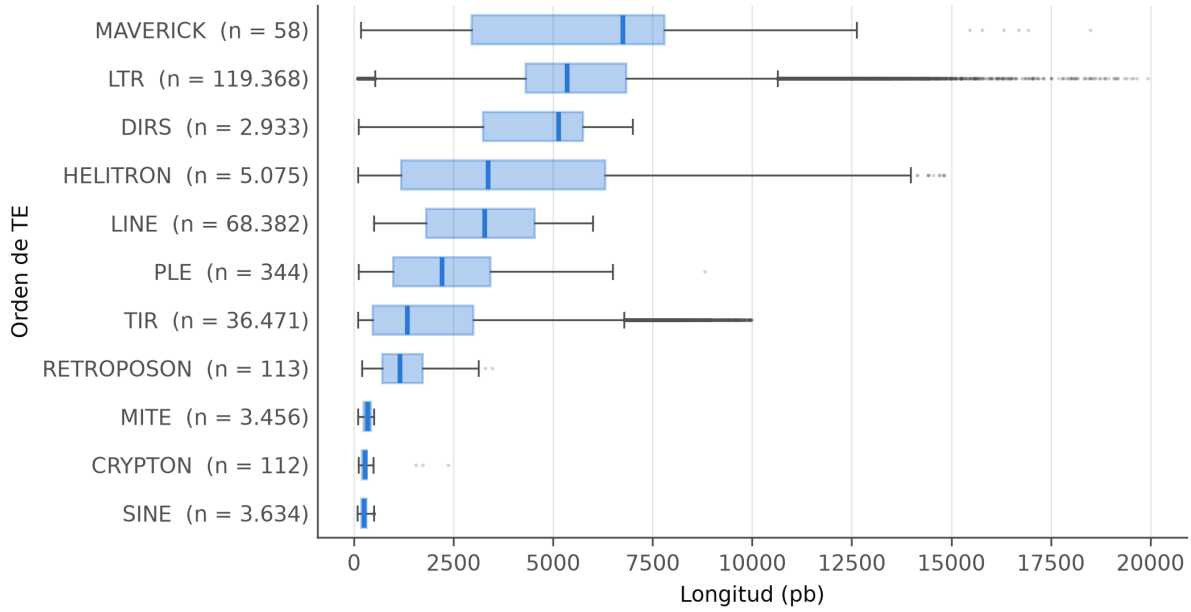


Figure 3. Distribución de la longitud (pb) de los TEs por orden, ordenados por mediana; *n* indica el número de secuencias de cada orden. Datos obtenidos de la colección `secuencias` en MongoDB.

Interpretación. La separación entre órdenes cortos y largos es coherente con su estructura: los SINE y los MITE son elementos no autónomos que no codifican las proteínas para movilizarse y por eso son cortos, mientras que los LTR, DIRS y MAVERICK contienen genes completos y superan con frecuencia los 5 kb. Los cortes exactos (500, 6 000 y 10 000 pb) no son biológicos: corresponden a los umbrales de longitud por superfamilia que PanTEon aplicó al construir la base para conservar solo secuencias con la estructura esperada [5]. Por lo tanto, la Figura 3 describe las secuencias retenidas por esos filtros y no la longitud natural de cada orden. Un caso que merece cautela es CRYPTON, cuya mediana (282 pb) es mucho menor de lo esperado para elementos que codifican una recombinasa, lo que sugiere que muchas de sus 112 secuencias son fragmentos. Finalmente, la amplia superposición entre órdenes como HELITRON, LINE y PLE muestra que la longitud por sí sola no basta para clasificar un TE. Para la base de datos, guardar `longitud` precalculada permite responder estas consultas con una proyección de dos campos, sin leer las secuencias.

Especies con más y menos TEs (Q4)

Pregunta. ¿Cuáles son las 10 especies con más TEs y las 10 con menos?

Resultado. Encabezan la lista *Malus domestica* (1 627), *Danio rerio* (1 352), *Callithrix jacchus* (1 294) y Homininae (1 173), seguidas de *Nicotiana attenuata*, *Colobus angolensis*, *Chlorocebus sabaeus*, *Geotrypetes seraphini*, *Sitophilus oryzae* y *Oryza sativa* (1 090). En el extremo opuesto, el mínimo es un solo TE y lo comparten 326 especies.

Interpretación. Las 10 especies con menos TEs no están bien definidas: con 326 empates en el mínimo, cualquier lista de diez es arbitraria y depende del criterio de desempate (aquí, orden alfabético). Es más informativo reportar cuántas especies tienen un solo TE. Por otro lado, el cuarto lugar del top lo ocupa Homininae, que no es una especie sino una subfamilia de primates; esto motivó la pregunta propia.

Pregunta propia 1: "especies" que son taxones superiores

Pregunta. ¿Cuántas entradas del campo especie corresponden en realidad a taxones superiores, cuántos TEs aportan y de qué órdenes son?

Resultado. Se identificaron 45 entradas cuyo nombre tiene una sola palabra (por ejemplo, Homininae, Myomorpha, Monotremata o Mammalia), todas del reino Animalia. Aportan 3 806 TEs (1,59 % de la base); Homininae (1 173) y Myomorpha (682) concentran casi la mitad. Por orden predominan TIR (1 920) y LTR (894).

Interpretación. Estas bibliotecas provienen de Dfam, donde las familias se asignan a un nodo de la taxonomía (un clado) y no necesariamente a una especie [9]. Por ello, el campo `especie` no siempre contiene una especie, lo que afecta a las consultas por especie (Q2 y Q4) y a la herencia de la taxonomía: estas entradas no tienen niveles inferiores (por ejemplo, familia) bien definidos. Se decidió conservarlas, porque eliminarlas haría perder 3 806 secuencias válidas, y documentarlas como una limitación del modelo.

Longitud de los TEs por clase, orden y superfamilia (Q5)

Pregunta. ¿Cuál es la longitud promedio, mínima y máxima de los TEs por clase, orden y superfamilia?

Resultado. La Clase I tiene 1 367 secuencias y una longitud promedio de 4 008,7 bases, mientras que la Clase II tiene 633 y un promedio de 1 258,9 bases. Sin embargo, la secuencia más larga pertenece a la Clase II, con 18 485 bases. Por orden, LTR tiene un promedio de 4 841,7 bases, mientras que SINE tiene uno de 220 bases. MAVERICK alcanza un promedio de 14 788,5 bases, pero solo tiene dos secuencias. Por superfamilia también hay diferencias: BELPAO tiene un promedio de 7 256,7 bases, GYPSY de 5 540,4 y TRNA de 216,9.

Interpretación. La longitud cambia bastante dependiendo de la clase, el orden y la superfamilia. También hay que tener en cuenta cuántas secuencias tiene cada grupo. Por ejemplo, el promedio de MAVERICK se calcula con solo dos registros, así que no representa tantos datos como el de LTR. Esta consulta permite comparar las longitudes sin tener que revisar las secuencias una por una.

Porcentaje de GC (Q6)

Pregunta. ¿Cuál es el porcentaje de GC promedio por orden y por reino?

Resultado. El porcentaje de GC cambia entre los órdenes. PLE tiene el promedio más alto, con 60,93 %, aunque solo cuenta con una secuencia. HELITRON alcanza el 60,13 % en 89 secuencias

y SINE el 55,96 % en 48. En cambio, LINE tiene un promedio de 41,67 %, TIR de 39,86 % y MAVERICK el más bajo, con 35,57 % en dos secuencias. Para el cálculo por reino, la consulta devuelve únicamente Animalia, con 2 000 secuencias y un porcentaje de GC promedio de 44,86 %.

Interpretación. Se observa que la proporción de G y C no es la misma en todos los órdenes. Sin embargo, algunos promedios se calculan con muy pocas secuencias, así que hay que tener cuidado al compararlos. Además, el resultado por reino no permite comparar Animalia con Plantae y Fungi, porque la consulta solo devolvió datos de Animalia.

TEs más largos y más cortos (Q7)

Pregunta. ¿Qué TEs son los más largos y los más cortos de cada reino, cuál es su clasificación y a qué especie pertenecen?

Resultado. En los resultados de la consulta solo aparece Animalia. El TE más largo es `Polinton-1_DR`, de la Clase II, orden MAVERICK y superfamilia MAVERICK, con 18 485 bases. Pertenece a *Danio rerio*. El más corto es `GalVar-1.2`, de la Clase I, orden SINE y superfamilia TRNA, con 97 bases, y pertenece a *Galeopterus variegatus*.

Interpretación. La diferencia entre ambos extremos es bastante grande. La consulta permite identificar estos casos y conocer su clasificación y especie. De todas formas, como solo aparece Animalia, no podemos decir cuáles son los TEs más largos y más cortos de Plantae y Fungi con estos resultados.

Búsqueda de una secuencia por su identificador (Q8)

Pregunta. ¿Cómo se puede buscar una secuencia por su nombre y mostrar su clasificación, especie y taxonomía completa?

Resultado. Al buscar la secuencia `PDB00000001`, se encontró que pertenece a la Clase II, al orden HELITRON y a la superfamilia HELITRON. Está asociada a *Halicephalobus mephisto*, que pertenece al reino Animalia, filo Nematoda, clase Chromadorea, orden Rhabditida y familia Panagrolaimidae.

Interpretación. Esta consulta sirve para buscar una secuencia por su identificador y conocer sus datos principales sin tener que mostrar toda la secuencia de ADN. Después se recupera la información de la especie para conocer su taxonomía completa. Así no es necesario repetir esos datos en cada documento de secuencia.

Secuencias de una superfamilia en una especie (Q9)

Pregunta. ¿Cómo se pueden listar todas las secuencias de una superfamilia en una especie determinada?

Resultado. Para la especie *Halicephalobus mephisto* y la superfamilia HELITRON se encontraron dos secuencias: `PDB00000001` y `PDB00000002`. En total, la consulta devolvió dos coincidencias.

Interpretación. Esta consulta permite filtrar las secuencias por especie y superfamilia, y mostrar sus identificadores en orden alfabético. También permite saber cuántos resultados se encontraron. Esto facilita revisar un grupo concreto de TEs sin tener que recorrer toda la colección.

Generación de bibliotecas (Q10 y Q11)

Pregunta. ¿Cómo se obtiene, desde la base de datos y no desde el archivo original, la biblioteca de una especie o de un reino dados?

Resultado. Una única función parametrizada por el nivel taxonómico (`especie` o `reino`) y su valor consulta primero `especies`, luego `secuencias` mediante el índice de `especie_id`, y escribe un archivo FASTA con el encabezado original (`>ID#CLASE/ORDEN/SUPERFAMILIA @Especie`) en la carpeta `library/`. La biblioteca de *Abortiporus biennis* contiene 1 secuencia. La del reino Fungi contiene 1 559 especies y 19 797 secuencias, el 8,25 % de las 239 946 de la base. El número de especies coincide con el de Fungi obtenido en Q3.

Interpretación. Cambiar solo el parámetro basta para generar la biblioteca de cualquier especie o reino sin leer el FASTA original. Esto muestra que la referencia `especie_id` permite construir bibliotecas de cualquier nivel taxonómico con dos consultas indexadas. El caso de *A. biennis* también ilustra la cola de la distribución descrita en Q4: hay especies con una sola secuencia.

Unión de secuencias con la taxonomía (Q12)

Pregunta. ¿Cómo se recupera la taxonomía de la especie de cada TE?

Resultado. Una consulta con `$lookup` sobre el campo `especie_id` devuelve, para cada TE, el reino, filo, clase, orden y familia de su especie. Por ejemplo, la secuencia `PumCon-1.11` (orden SINE) se une a *Puma concolor*: Animalia, Chordata, Mammalia, Carnivora, Felidae. Las secuencias `PDB00000001` y `PDB00000002` (HELITRON) se unen a *Halicephalobus mephisto* (Animalia, Nematoda, Chromadorea, Rhabditida, Panagrolaimidae).

Interpretación. Una sola referencia recupera toda la taxonomía, que se almacena una vez en 2 790 documentos y no se repite en las 239 946 secuencias. Los identificadores no siguen todos el formato `PDB########` (existen identificadores como `PumCon-1.11`), por lo que `pdb_id` se trata como una cadena arbitraria y única.

Comparación por origen de la biblioteca (Q13)

Pregunta. Agrupando las especies según la columna `Source` del CSV, ¿cuántas especies y secuencias hay en cada grupo, cuál es el promedio de secuencias por especie y cuál la longitud promedio de los TEs?

Resultado. La Tabla 4 resume la comparación.

Table 4. Comparación entre especies con biblioteca Dfam y RM2+MCH.

Origen	Especies	Secuencias	Secuencias/especie	Longitud media (pb)
Dfam	386	31 592	81,8	1 653,8
RM2+MCH	2 404	208 354	86,7	4 375,5

Las especies con biblioteca RM2+MCH son el 86,2 % y aportan el 86,8 % de las secuencias. El número de secuencias por especie es similar entre orígenes (86,7 frente a 81,8, una diferencia del 6 %), pero las secuencias de RM2+MCH son 2,6 veces más largas (4 375 frente a 1 654 pb).

Interpretación. Los dos orígenes no son equivalentes en longitud, y cualquier análisis que compare especies de ambos grupos debe tenerlo en cuenta. La causa no se midió: puede deberse a la composición de órdenes de cada grupo (los órdenes largos y cortos difieren mucho en longitud, según la sección anterior) o a la forma de construir cada biblioteca. La longitud media es un promedio ponderado por secuencia.

Control de calidad (Q14)

Pregunta. ¿Cuántas secuencias contienen caracteres distintos de A, C, G y T, y cuántas posiciones se descartan por ello?

Resultado. La consulta encontró 54 837 secuencias (22,85 %) con caracteres distintos de A, C, G o T. Todas tenían `porcentaje_n` > 0, de modo que ese campo detecta todas las secuencias afectadas. Se descartaron 2 147 161 posiciones, el 0,223 % de las 963 901 415 bases, unas 39 por secuencia afectada.

Interpretación. Casi una de cada cuatro secuencias tiene al menos un carácter ambiguo, pero la pérdida de información es mínima. Descartar solo las ventanas afectadas, y no secuencias enteras, evita eliminar 54 837 TEs. La política queda justificada, y las frecuencias siguen normalizadas por `total_ventanas`. El control no distingue si los caracteres no válidos son siempre N o incluyen otros códigos IUPAC.

Perfil de *k*-mers de una secuencia (Q15)

Pregunta. ¿Cuál es el perfil de k-mers de una secuencia dada para un k dado?

Resultado. La función recibe el identificador de la secuencia y k, busca la secuencia por `pdb_id` (índice único) y recupera su documento por (`secuencia_id`, `k`), también indexado. Para `PDB00000001` (GC = 31,5 %) con $k = 3$ hay 1 989 ventanas válidas. Los diez 3-mers más frecuentes suman 771 ventanas, el 38,8 % del total. Ocho de ellos están formados solo por A y T (AAA, TTT, AAT, ATT, TAT, ATA, TAA y TTA) y los otros dos (GAA y ATG) contienen una sola G. El más frecuente, AAA, representa el 7,2 % de las ventanas, frente al 4,9 % de TTT.

Interpretación. El perfil es coherente con el bajo contenido de GC de la secuencia, lo que indica que los conteos almacenados reproducen su composición. Con una sola secuencia no es posible generalizar la asimetría entre AAA y TTT.

Perfiles de dinucleótidos por orden de TE

Pregunta. ¿Hay órdenes de TE con perfiles de 2-mers parecidos o muy distintos?

Resultado. La Figura 4 muestra la frecuencia promedio de los 16 dinucleótidos en cada uno de los 11 órdenes. El promedio se calcula sobre la frecuencia de cada secuencia, de modo que una secuencia larga no pesa más que una corta; las 16 frecuencias de cada orden suman 1. Los órdenes más parecidos, medidos por distancia euclidiana entre perfiles, son HELITRON–TIR (0,0151), MITE–TIR (0,0159) y LINE–LTR (0,0168). Los más distintos son MAVERICK–SINE (0,1076), HELITRON–SINE (0,0977) y SINE–TIR (0,0975). SINE aparece en cuatro de los cinco pares más alejados: tiene TT bajo (0,048 frente a 0,082–0,104 en HELITRON, LINE, LTR, MAVERICK, MITE y TIR), AT bajo (0,042), y GG y CC altos (0,090 y 0,076). La separación entre extremos es unas siete veces la distancia entre los órdenes más parecidos. CG es el dinucleótido menos frecuente en casi todos los órdenes, con un mínimo de 0,030 en LINE y LTR.

Interpretación. Los perfiles de 2-mers distinguen órdenes: TIR, MITE, HELITRON y MAVERICK comparten AA y TT altos y CG bajo, LINE y LTR forman otro par muy parecido, y SINE se aparta del resto. El número de secuencias por orden es muy desigual (de 58 en MAVERICK a 119 368 en LTR), por lo que los promedios de MAVERICK, CRYPTON ($n = 112$) y RETROPOSON ($n = 113$) son menos fiables que los de LTR, LINE o TIR.

Comparación de estrategias de almacenamiento de los *k*-mers

Se compararon dos formas de guardar los perfiles de k-mers: la estrategia 1, una colección aparte con un documento por secuencia y por k, y la estrategia 2, un documento por secuencia que embebe los seis perfiles en un arreglo. Ambas usan la misma estructura de `conteos`. El experimento se hizo con una muestra de 5 000 secuencias (30 000 documentos en la estrategia 1 y 5 000 en la 2) en un equipo con Windows 11, procesador Intel de 12 núcleos lógicos, 8 GB de RAM, Python 3.12.3 y MongoDB 9.0.4. Los k-mers se calcularon por lotes de 25 secuencias fuera

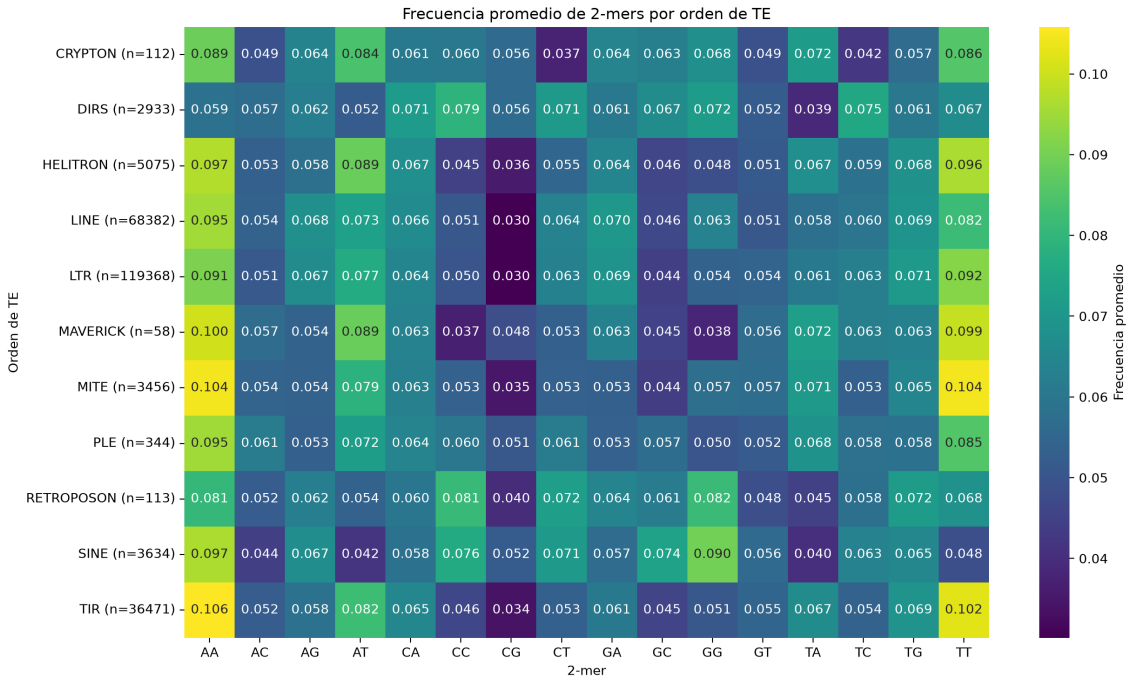


Figure 4. Frecuencia promedio de los 16 2-mers por orden de TE (promedio de las frecuencias por secuencia; n es el número de secuencias del orden).

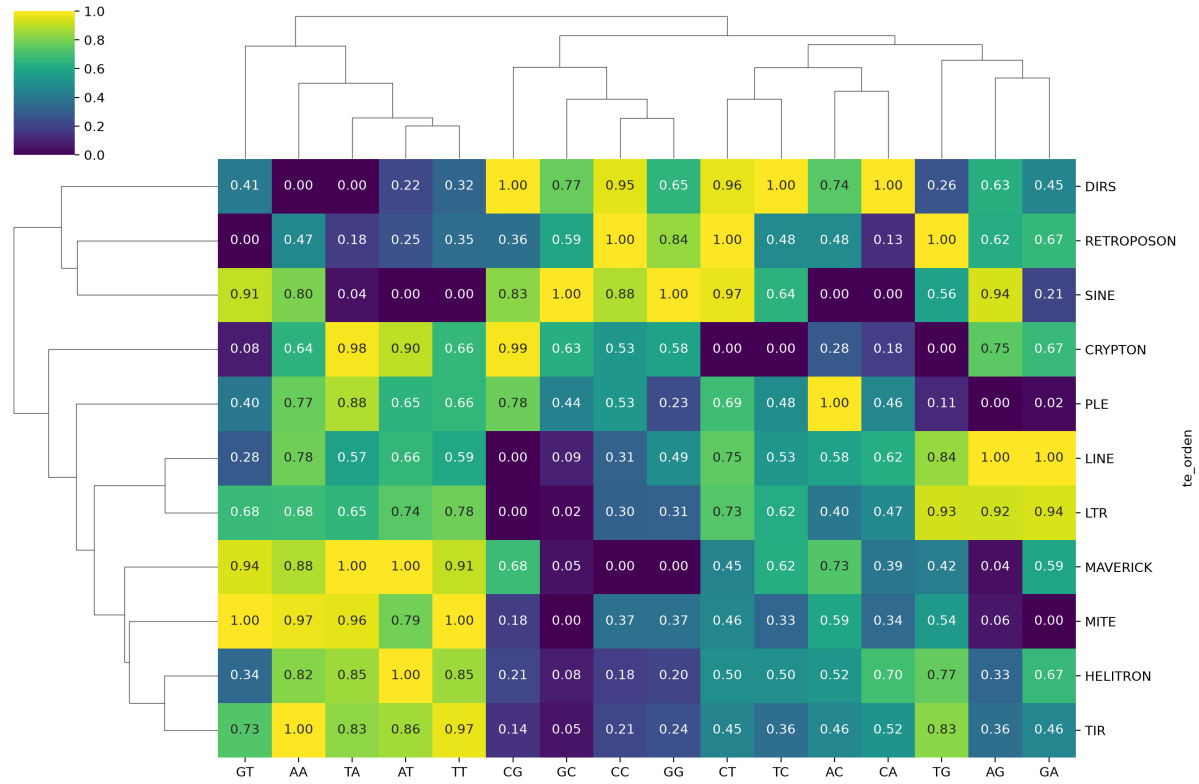


Figure 5. Agrupamiento jerárquico de los órdenes. Los valores están reescalados por columna (0 a 1), por lo que los extremos 0 y 1 no son frecuencias reales; las frecuencias reales están en la Figura 4.

del cronómetro, de modo que solo se midió la inserción, y la carga se repitió tres veces.

Table 5. Carga y espacio de las dos estrategias (muestra de 5 000 secuencias; tiempo como media ± desviación estándar de tres repeticiones).

	Estrategia 1 (colección aparte)	**Estrategia 2** (embebida)
Documentos	30 000	5 000
Tiempo de inserción (s)	2,74 ± 1,13	2,26 ± 0,94
Datos sin comprimir (MB)	134,41	133,05
Almacenamiento en disco (MB)	42,56	45,98
Índices (MB)	1,10	0,48
Tamaño medio por documento (KB)	4,48	26,61

Table 6. Tiempo de tres consultas representativas sin y con índices (mínimo de tres ejecuciones, `explain("executionStats")`).

		Sin índice		**Con índice**	
Consulta	**Estrategia**	ms	docs exam.	ms	docs exam.
C1: perfil $k = 6$ de una secuencia	1	74	30 000	0	1
	2	11	5 000	0	1
C2: perfiles $k = 2$ de un orden (133)	1	59	30 000	1	133
	2	13	5 000	3	133
C3: $k = 6$ y más de 1 000 ventanas (3 005)	1	51	30 000	14	3 005
	2	118	5 000	148	3 005

Carga. La estrategia 2 tardó en promedio 2,26 s frente a 2,74 s de la estrategia 1 (Tabla 5), pero la diferencia (0,48 s) es menor que la desviación estándar de cada una, y la tercera repetición fue más lenta en ambas estrategias y también en el cálculo de los k-mers (22,2 s frente a cerca de 10 s), probablemente por la carga del equipo en ese momento. Con estos datos no se puede afirmar que una estrategia se cargue más rápido que la otra. Como referencia, el cálculo y la carga de los 1 439 676 documentos de la base completa con la estrategia 1 tomaron cerca de 1 h 20 min en otro equipo del grupo.

Espacio. Los datos sin comprimir son casi iguales (134 frente a 133 MB), lo que era de esperar porque se guardan los mismos conteos. En disco, la estrategia 1 ocupó un 7 % menos (42,6 frente a 46,0 MB), aunque sus índices pesan más (1,10 frente a 0,48 MB) porque indexan seis veces más documentos. La diferencia principal está en el tamaño de cada documento: 4,5 KB frente a 26,6 KB, ya que en la estrategia 2 cada documento contiene los seis perfiles.

Consultas. Sin índices, cada consulta recorre la colección completa, y la estrategia 2 fue más rápida en C1 y C2 porque examina seis veces menos documentos (5 000 frente a 30 000). Con índices, las dos estrategias son prácticamente iguales en C1 (0 ms) y muy cercanas en C2 (1 frente a 3 ms). La diferencia aparece en C3, que filtra por valores dentro del perfil: la estrategia 1 tardó 14 ms con índice, mientras que la estrategia 2 tardó 148 ms, incluso más que sin índice (118 ms). En la estrategia 2, el índice sobre los campos del arreglo encuentra los documentos, pero cada uno pesa unos 27 KB y se debe leer completo para devolver solo el perfil que cumple la condición.

Qué estrategia conviene. Para el uso de este proyecto, en el que las consultas trabajan con un valor de k a la vez (el perfil de una secuencia para un k, la figura de 2-mers por orden o filtros sobre los perfiles), la estrategia 1 es la más adecuada: con índices iguala o supera a la

embebida, ocupa algo menos en disco y mantiene documentos pequeños. La estrategia 2 solo resulta ventajosa cuando no hay índices y se consulta el perfil de pocas secuencias, porque recorre menos documentos. Estos resultados corresponden a una muestra de 5 000 secuencias en un solo equipo; con la base completa los tiempos sin índice crecerían con el tamaño de la colección, lo que refuerza la necesidad de los índices.

4 Discussion

5 Discusión

Diseño del modelo. Las referencias (`especie_id`, `secuencia_id`) evitan repetir la taxonomía en 239 946 documentos y permiten corregir un dato taxonómico modificando un único documento. El costo es que las consultas por taxonomía (Q2, Q3, Q12 y Q13) requieren un `$lookup`, que se mitigó agrupando antes de unir. Copiar `te_orden` en `kmers` es una desnormalización deliberada: el costo es un campo redundante y el beneficio es filtrar por orden y por k sin unir colecciones, como exige el mapa de calor. Un documento por par (secuencia, k) mantiene cada documento pequeño (como máximo 4 096 claves) y permite consultar un solo k sin leer los demás. Una alternativa es un único documento por secuencia con los seis k embebidos, que reduciría el número de documentos a 239 946 pero obligaría a leer o proyectar todo el documento para cada consulta; en este trabajo no se midieron ambas alternativas.

Composición y calidad de los datos. La base está sesgada hacia los retrotransposones LTR y LINE, que suman el 78,2 % de las secuencias, y muchas especies tienen pocos TEs (326 tienen uno solo). Además, las longitudes reflejan los filtros aplicados por PanTEon y no la longitud natural de cada orden. El 22,85 % de las secuencias contiene algún carácter ambiguo, pero solo el 0,223 % de las bases se descarta; por eso descartar ventanas y no secuencias completas conserva casi toda la información. Las bibliotecas Dfam y RM2+MCH tienen un número parecido de secuencias por especie, pero difieren 2,6 veces en longitud, por lo que no deben tratarse como equivalentes.

Lectura biológica de los perfiles. SINE es el orden más corto (mediana 254 pb) y también el de perfil de dinucleótidos más singular; este trabajo no evaluó si ambas observaciones están relacionadas. La baja frecuencia de CG en casi todos los órdenes es coherente con la pérdida de dinucleótidos CpG por metilación y mutación, aunque este trabajo no midió esos mecanismos. Los resultados sugieren que los k-mers pueden servir como firma para clasificar TEs, en línea con su uso en las herramientas revisadas por PanTEon [5]. La longitud sola no basta: HELITRON, LINE y PLE se superponen, mientras que sus perfiles de 2-mers sí los distinguen en parte.

Limitaciones. (i) El desbalance entre órdenes (58 a 119 368 secuencias) hace desiguales los promedios. (ii) El promedio por orden oculta la variación entre especies y entre secuencias; no se calculó dispersión. (iii) Las distancias se calcularon solo con $k = 2$. (iv) Los resultados dependen de los metadatos y las etiquetas de PanTEon v1.6.1, incluidas las superfamilias sin determinar y los identificadores de especie que corresponden a taxones superiores. (v) No se comparó experimentalmente el costo de distintas estrategias de almacenamiento de los k-mers. (vi) Los resultados se obtuvieron en la base de trabajo `panteon_prueba`, que contiene la carga completa de las 239 946 secuencias.

6 Conclusions

7 Conclusiones

Se diseñó e implementó una base documental en MongoDB que integra 239 946 secuencias de TEs, 2 790 especies y 1 439 676 documentos de k-mers ($k = 1$ a 6). La base permite generar bibliotecas por especie o reino (por ejemplo, 19 797 secuencias para Fungi), relacionar secuencias con su taxonomía y recuperar el perfil de k-mers de cualquier secuencia mediante una consulta indexada. Las consultas mostraron una base fuertemente desbalanceada (LTR, LINE y TIR concentran el 93,4 % de las secuencias), con composiciones de órdenes distintas entre reinos y longitudes que dependen de los filtros de PanTEon. El control de calidad mostró que el 22,85 % de las secuencias contiene caracteres ambiguos, pero solo el 0,223 % de las bases, por lo que descartar ventanas, en lugar de secuencias, conserva casi toda la información. Los perfiles de dinucleótidos distinguen órdenes de TE: SINE es el más singular, y HELITRON, MITE y TIR los más parecidos entre sí. Como trabajo futuro, conviene comparar experimentalmente estrategias de almacenamiento de los k-mers (tiempo de carga, espacio y tiempo de consulta), incluir medidas de dispersión por orden, usar perfiles con $k > 2$, y emplear los k-mers almacenados para entrenar y evaluar clasificadores.

List of abbreviations

Declarations

Disponibilidad de datos y materiales

Los datos utilizados provienen de PanTEon Database v1.6.1, disponible públicamente en Zenodo bajo licencia CC BY 4.0 (`https://zenodo.org/records/19372616`) [6]; se usaron los archivos `PanTEon_Database_v1.6.1.fasta` y `PanTEon_Database_metadata_v1.6.1.csv`. El código del *pipeline*, las consultas, los experimentos y las figuras está disponible en el repositorio del equipo: `https://github.com/ManuIngSistem12/Elementos-Transponibles-TEs-`. El repositorio incluye un archivo `README.md` con las instrucciones para reproducir los resultados desde cero (instalación de dependencias con `requirements.txt`, descarga de los datos, ejecución de MongoDB local y orden de ejecución de los *notebooks*) y un respaldo de la base de datos generada.

References

[1] Bourque G, Burns KH, Gehring M, Gorbunova V, Seluanov A, Hammell M, et al. Ten things you should know about transposable elements. Genome Biol. 2018;19:199. doi:10.1186/s13059-018-1577-z.

[2] International Human Genome Sequencing Consortium. Initial sequencing and analysis of the human genome. Nature. 2001;409(6822):860–921. doi:10.1038/35057062.

[3] Schnable PS, Ware D, Fulton RS, Stein JC, Wei F, et al. The B73 maize genome: complexity, diversity, and dynamics. Science. 2009;326(5956):1112–5. doi:10.1126/science.1178534.

[4] Wicker T, Sabot F, Hua-Van A, Bennetzen JL, Capy P, Chalhoub B, et al. A unified classification system for eukaryotic transposable elements. Nat Rev Genet. 2007;8(12):973–82. doi:10.1038/nrg2165.

[5] Orozco-Arias S, Ferrer-Pomer I, Rodrigues de Goes F, Gaviria-Orrego S, Gómiz-Fernández J, Llatser-Torres J, et al. PanTEon: a cross-kingdom framework to guide the design of transposable element classifiers. bioRxiv. 2026. doi:10.64898/2026.04.01.715927.

[6] Orozco-Arias S, et al. PanTEon Database v1.6.1 [Dataset]. Zenodo; 2026. https://zenodo.org/records/19372616.

[7] Flynn JM, Hubley R, Goubert C, Rosen J, Clark AG, Feschotte C, et al. RepeatModeler2 for automated genomic discovery of transposable element families. Proc Natl Acad Sci USA. 2020;117(17):9451–7. doi:10.1073/pnas.1921046117.

[8] Orozco-Arias S, Sierra P, Durbin R, González J. MCHelper automatically curates transposable element libraries across eukaryotic species. Genome Res. 2024;34(12):2256–68. doi:10.1101/gr.278821.123.

[9] Storer J, Hubley R, Rosen J, Wheeler TJ, Smit AF. The Dfam community resource of transposable element families, sequence models, and genome annotations. Mob DNA. 2021;12:2. doi:10.1186/s13100-020-00230-y.